Search models, datasets, users...

Transformers documentation
Donut

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

This model was released on 2021-11-30 and added to Hugging Face Transformers on 2022-08-12.

Donut

PyTorch

Copy page

Donut (Document Understanding Transformer) is a visual document understanding model that doesn't require an Optical Character Recognition (OCR) engine. Unlike traditional approaches that extract text using OCR before processing, Donut employs an end-to-end Transformer-based architecture to directly analyze document images. This eliminates OCR-related inefficiencies making it more accurate and adaptable to diverse languages and formats.

Donut features vision encoder (Swin) and a text decoder (BART). Swin converts document images into embeddings and BART processes them into meaningful text sequences.

You can find all the original Donut checkpoints under the Naver Clova Information Extraction organization.

> Click on the Donut models in the right sidebar for more examples of how to apply Donut to different language and vision tasks.

The examples below demonstrate how to perform document understanding tasks using Donut with Pipeline and AutoModel

Pipeline AutoModel

Ask HuggingChat

```
# pip install datasets
import torch
from transformers import pipeline
from PIL import Image

pipeline = pipeline(
    task="document-question-answering",
    model="naver-clova-ix/donut-base-finetuned-docvqa",
    device=0,
    dtype=torch.float16
)
dataset = load_dataset("hf-internal-testing/example-documents", split="test")
image = dataset[0]["image"]

pipeline(image=image, question="What time is the coffee break?")
```

Quantization reduces the memory burden of large models by representing the weights in a lower precision. Refer to the Quantization overview for more available quantization backends.

The example below uses torchao to only quantize the weights to int4.

```
# pip install datasets torchao
import torch
from datasets import load_dataset
from transformers import TorchAoConfig, AutoProcessor, AutoModelForImageTextToText

quantization_config = TorchAoConfig("int4_weight_only", group_size=128)
processor = AutoProcessor.from_pretrained("naver-clova-ix/donut-base-finetuned-docvqa"
model = AutoModelForImageTextToText.from_pretrained("naver-clova-ix/donut-base-finetun

dataset = load_dataset("hf-internal-testing/example-documents", split="test")
image = dataset[0]["image"]
question = "What time is the coffee break?"
task_prompt = f"<s_docvqa><s_question>{question}</s_question><s_answer>"
inputs = processor(image, task_prompt, return_tensors="pt")

outputs = model.generate(
    input_ids=inputs.input_ids,
    pixel_values=inputs.pixel_values,
    max_length=512
)
```

```
answer = processor.decode(outputs[0], skip_special_tokens=True)
print(answer)
```

Notes

- Use Donut for document image classification as shown below.

```
>>> import re
>>> from transformers import DonutProcessor, VisionEncoderDecoderModel
>>> from accelerate import Accelerator
>>> from datasets import load_dataset
>>> import torch

>>> processor = DonutProcessor.from_pretrained("naver-clova-ix/donut-base-finetune
>>> model = VisionEncoderDecoderModel.from_pretrained("naver-clova-ix/donut-base-f

>>> device = Accelerator().device
>>> model.to(device)  # doctest: +IGNORE_RESULT

>>> # load document image
>>> dataset = load_dataset("hf-internal-testing/example-documents", split="test")
>>> image = dataset[1]["image"]

>>> # prepare decoder inputs
>>> task_prompt = "<s_rvlcdip>"
>>> decoder_input_ids = processor.tokenizer(task_prompt, add_special_tokens=False,

>>> pixel_values = processor(image, return_tensors="pt").pixel_values

>>> outputs = model.generate(
...     pixel_values.to(device),
...     decoder_input_ids=decoder_input_ids.to(device),
...     max_length=model.decoder.config.max_position_embeddings,
...     pad_token_id=processor.tokenizer.pad_token_id,
...     eos_token_id=processor.tokenizer.eos_token_id,
...     use_cache=True,
...     bad_words_ids=[[processor.tokenizer.unk_token_id]],
...     return_dict_in_generate=True,
... )

>>> sequence = processor.batch_decode(out
>>> sequence = sequence.replace(processor
>>> sequence = re.sub(r"<.*?>", "", sequence, count=1).strip()  # remove first tas
```

```
>>> print(processor.token2json(sequence))
{'class': 'advertisement'}
```

- Use Donut for document parsing as shown below.

```
>>> import re
>>> from accelerate import Accelerator
>>> from datasets import load_dataset
>>> from transformers import DonutProcessor, VisionEncoderDecoderModel
>>> import torch

>>> processor = DonutProcessor.from_pretrained("naver-clova-ix/donut-base-finetune
>>> model = VisionEncoderDecoderModel.from_pretrained("naver-clova-ix/donut-base-f

>>> device = Accelerator().device
>>> model.to(device)  # doctest: +IGNORE_RESULT

>>> # load document image
>>> dataset = load_dataset("hf-internal-testing/example-documents", split="test")
>>> image = dataset[2]["image"]

>>> # prepare decoder inputs
>>> task_prompt = "<s_cord-v2>"
>>> decoder_input_ids = processor.tokenizer(task_prompt, add_special_tokens=False,

>>> pixel_values = processor(image, return_tensors="pt").pixel_values

>>> outputs = model.generate(
...     pixel_values.to(device),
...     decoder_input_ids=decoder_input_ids.to(device),
...     max_length=model.decoder.config.max_position_embeddings,
...     pad_token_id=processor.tokenizer.pad_token_id,
...     eos_token_id=processor.tokenizer.eos_token_id,
...     use_cache=True,
...     bad_words_ids=[[processor.tokenizer.unk_token_id]],
...     return_dict_in_generate=True,
... )

>>> sequence = processor.batch_decode(outputs.sequences)[0]
>>> sequence = sequence.replace(processor.tokenizer.eos_token, "").replace(process
>>> sequence = re.sub(r"<.*?>", "", sequence, count=1).strip()  # remove first tas
>>> print(processor.token2json(sequence))
```

```
{'menu': {'nm': 'CINNAMON SUGAR', 'unitprice': '17,000', 'cnt': '1 x', 'price': '1
{'total_price': '17,000', 'cashprice': '20,000', 'changeprice': '3,000'}}
```

DonutSwinConfig

class transformers.**DonutSwinConfig**

<>

```
( image_size = 224, patch_size = 4, num_channels = 3, embed_dim = 96, depths = [2, 2, 6,
2], num_heads = [3, 6, 12, 24], window_size = 7, mlp_ratio = 4.0, qkv_bias = True,
hidden_dropout_prob = 0.0, attention_probs_dropout_prob = 0.0, drop_path_rate = 0.1,
hidden_act = 'gelu', use_absolute_embeddings = False, initializer_range = 0.02,
layer_norm_eps = 1e-05, **kwargs )
```

Expand 16 parameters

This is the configuration class to store the configuration of a DonutSwinModel. It is used to instantiate a Donut model according to the specified arguments, defining the model architecture. Instantiating a configuration with the defaults will yield a similar configuration to that of the Donut naver-clova-ix/donut-base architecture.

Configuration objects inherit from PreTrainedConfig and can be used to control the model outputs. Read the documentation from PreTrainedConfig for more information.

Example:

```
>>> from transformers import DonutSwinConfig, DonutSwinModel

>>> # Initializing a Donut naver-clova-ix/donut-base style configuration
>>> configuration = DonutSwinConfig()

>>> # Randomly initializing a model from the naver-clova-ix/donut-base style configu
>>> model = DonutSwinModel(configuration)

>>> # Accessing the model configuration
>>> configuration = model.config
```

DonutImageProcessor

class transformers.**DonutImageProcessor** <>

```
( do_resize: bool = True, size: dict[str, int] | None = None, resample: Resampling =
<Resampling.BILINEAR: 2>, do_thumbnail: bool = True, do_align_long_axis: bool = False,
do_pad: bool = True, do_rescale: bool = True, rescale_factor: int | float =
0.00392156862745098, do_normalize: bool = True, image_mean: float | list[float] | None =
None, image_std: float | list[float] | None = None, **kwargs )
```

Expand 11 parameters

Constructs a Donut image processor.

preprocess

<>

```
( images: typing.Union[ForwardRef('PIL.Image.Image'), numpy.ndarray,
ForwardRef('torch.Tensor'), list['PIL.Image.Image'], list[numpy.ndarray],
list['torch.Tensor']], do_resize: bool | None = None, size: dict[str, int] | None =
None, resample: PIL.Image.Resampling | None = None, do_thumbnail: bool | None = None,
do_align_long_axis: bool | None = None, do_pad: bool | None = None, random_padding: bool
= False, do_rescale: bool | None = None, rescale_factor: float | None = None,
do_normalize: bool | None = None, image_mean: float | list[float] | None = None,
image_std: float | list[float] | None = None, return_tensors: str |
transformers.utils.generic.TensorType | None = None, data_format:
transformers.image_utils.ChannelDimension | None = <ChannelDimension.FIRST:
'channels_first'>, input_data_format: str | transformers.image_utils.ChannelDimension |
None = None )
```

Expand 16 parameters

Preprocess an image or batch of images.

DonutImageProcessorFast

class transformers.**DonutImageProcessorFast**

<>

```
( **kwargs:
typing_extensions.Unpack[transformers.models.donut.image_processing_donut.DonutImageProcessorKwargs] )
```

Parameters

- **do_thumbnail** (`bool`, *kwargs*, *optional*, defaults to `self.do_thumbnail`) — Whether to resize the image using thumbnail method.
- **do_align_long_axis** (`bool`, *kwargs*, *optional*, defaults to `self.do_align_long_axis`) — Whether to align the long axis of the image with the long axis of `size` by rotating by 90 degrees.
- ***kwargs** (ImagesKwargs, *optional*) — Additional image preprocessing options. Model-specific kwargs are listed above; see the TypedDict class for the complete list of supported arguments.

Constructs a image_processor_class image processor.

preprocess <>

```
( images: typing.Union[ForwardRef('PIL.Image.Image'), numpy.ndarray,
ForwardRef('torch.Tensor'), list['PIL.Image.Image'], list[numpy.ndarray],
list['torch.Tensor']], **kwargs:
typing_extensions.Unpack[transformers.models.donut.image_processing_donut.DonutImageProc
essorKwargs] ) → ~image_processing_base.BatchFeature
```

Expand 5 parameters

DonutProcessor

class transformers.DonutProcessor

<>

```
( image_processor = None, tokenizer = None, **kwargs )
```

Parameters

- **image_processor** (`image_processor_class`) — The image processor is a required input.
- **tokenizer** (`tokenizer_class`) — The tokenizer is a required input.

Constructs a DonutProcessor which wraps a image processor and a tokenizer into a single processor.

DonutProcessor offers all the functionalities of `image_processor_class` and `tokenizer_class`. See the `~image_processor_class` and `~tokenizer_class` for more information.

__call__

<>

```
( images: typing.Union[ForwardRef('PIL.Image.Image'), numpy.ndarray,
ForwardRef('torch.Tensor'), list['PIL.Image.Image'], list[numpy.ndarray],
list['torch.Tensor'], NoneType] = None, text: str | list[str] | None = None, **kwargs:
typing_extensions.Unpack[transformers.models.donut.processing_donut.DonutProcessorKwargs
] )
```

Parameters

- **images** (`Union[PIL.Image.Image, numpy.ndarray, torch.Tensor, list[PIL.Image.Image], list[numpy.ndarray], list[torch.Tensor]]`, *optional*) — Image to preprocess. Expects a single or batch of images with pixel values ranging from 0 to 255. If passing in images with pixel values between 0 and 1, set `do_rescale=False`.
- **text** (`Union[str, list[str]]`, *optional*) — The sequence or batch of sequences to be encoded. Each sequence can be a string or a list of strings (pretokenized string). If you pass a pretokenized input, set `is_split_into_words=True` to avoid ambiguity with batched inputs.
- **return_tensors** (`str` or TensorType, *optional*) — If set, will return tensors of a particular framework. Acceptable values are:
  - `'pt'`: Return PyTorch `torch.Tensor` objects.
  - `'np'`: Return NumPy `np.ndarray` obje

- ***kwargs** (ProcessingKwargs, *optional*) — Additional processing options for each modality (text, images, videos, audio). Model-specific parameters are listed above; see the TypedDict class for the complete list of supported arguments.

from_pretrained

```
( pretrained_model_name_or_path: str | os.PathLike, cache_dir: str | os.PathLike | None = None, force_download: bool = False, local_files_only: bool = False, token: str | bool | None = None, revision: str = 'main', **kwargs )
```

Parameters

- **pretrained_model_name_or_path** (`str` or `os.PathLike`) — This can be either:
  - a string, the *model id* of a pretrained feature_extractor hosted inside a model repo on huggingface.co.
  - a path to a *directory* containing a feature extractor file saved using the save_pretrained() method, e.g., `./my_model_directory/`.
  - a path or url to a saved feature extractor JSON *file*, e.g., `./my_model_directory/preprocessor_config.json`.
- ***kwargs** — Additional keyword arguments passed along to both from_pretrained() and `~tokenization_utils_base.PreTrainedTokenizer.from_pretrained`.

Instantiate a processor associated with a pretrained model.

> This class method is simply calling the feature extractor from_pretrained(), image processor ImageProcessingMixin and the tokenizer `~tokenization_utils_base.PreTrainedTokenizer.from_pretrained` methods. Please refer to the docstrings of the methods above for more information.

save_pretrained

```
( save_directory, push_to_hub: bool = False, **kwargs )
```

Parameters

- **save_directory** (`str` or `os.PathLike`) — Directory where the feature extractor JSON file and the tokenizer files will be saved (director

- **push_to_hub** (`bool`, *optional*, defaults to `False`) — Whether or not to push your model to the Hugging Face model hub after saving it. You can specify the repository you want to push to with `repo_id` (will default to the name of `save_directory` in your namespace).
- **kwargs** (`dict[str, Any]`, *optional*) — Additional key word arguments passed along to the push_to_hub() method.

Saves the attributes of this processor (feature extractor, tokenizer…) in the specified directory so that it can be reloaded using the from_pretrained() method.

> This class method is simply calling save_pretrained() and save_pretrained(). Please refer to the docstrings of the methods above for more information.

batch_decode <>

```
( *args, **kwargs )
```

This method forwards all its arguments to PreTrainedTokenizer's batch_decode(). Please refer to the docstring of this method for more information.

decode <>

```
( *args, **kwargs )
```

This method forwards all its arguments to PreTrainedTokenizer's decode(). Please refer to the docstring of this method for more information.

DonutSwinModel

class transformers.**DonutSwinModel** <>

```
( config, add_pooling_layer = True, use_mask_token = False )
```

Parameters

- **config** (DonutSwinModel) — Model configuratio
Initializing with a config file does not load the we

configuration. Check out the from_pretrained() method to load the model weights.

- **add_pooling_layer** (`bool`, *optional*, defaults to `True`) — Whether to add a pooling layer
- **use_mask_token** (`bool`, *optional*, defaults to `False`) — Whether to use a mask token for masked image modeling.

The bare Donut Swin Model outputting raw hidden-states without any specific head on top.

This model inherits from PreTrainedModel. Check the superclass documentation for the generic methods the library implements for all its model (such as downloading or saving, resizing the input embeddings, pruning heads etc.)

This model is also a PyTorch torch.nn.Module subclass. Use it as a regular PyTorch Module and refer to the PyTorch documentation for all matter related to general usage and behavior.

forward

<>

```
( pixel_values: torch.FloatTensor | None = None, bool_masked_pos: torch.BoolTensor | None = None, output_attentions: bool | None = None, output_hidden_states: bool | None = None, interpolate_pos_encoding: bool = False, return_dict: bool | None = None, **kwargs ) → DonutSwinModelOutput or tuple(torch.FloatTensor)
```

Expand 6 parameters

The DonutSwinModel forward method, overrides the `__call__` special method.

> Although the recipe for forward pass needs to be defined within this function, one should call the `Module` instance afterwards instead of this since the former takes care of running the pre and post processing steps while the latter silently ignores them.

- **last_hidden_state** (`torch.FloatTensor` of shape `(batch_size, sequence_length, hidden_size)`, *optional*, defaults to `None`) — Sequence of hidden-states at the output of the last layer of the model.

- **pooler_output** (`torch.FloatTensor` of shape `(batch_size, hidden_size)`, *optional*, returned when `add_pooling_layer=True` is passed) — Average pooling of the last layer hidden-state.

- **hidden_states** (`tuple[torch.FloatTensor, ...]`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) — Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the model has an embedding layer, + one for the output of each layer) of shape `(batch_size, sequence_length, hidden_size)`.

  Hidden-states of the model at the output of each layer plus the optional initial embedding outputs.

- **attentions** (`tuple[torch.FloatTensor, ...]`, *optional*, returned when `output_attentions=True` is passed or when `config.output_attentions=True`) — Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size, num_heads, sequence_length, sequence_length)`.

  Attentions weights after the attention softmax, used to compute the weighted average in the self-attention heads.

- **reshaped_hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) — Tuple of `torch.FloatTensor` (one for the output of the embeddings + one for the output of each stage) of shape `(batch_size, hidden_size, height, width)`.

  Hidden-states of the model at the output o
  outputs reshaped to include the spatial di

DonutSwinForImageClassification

class transformers.**DonutSwinForImageClassification** <>

```
( config )
```

Parameters

- **config** (DonutSwinForImageClassification) — Model configuration class with all the parameters of the model. Initializing with a config file does not load the weights associated with the model, only the configuration. Check out the from_pretrained() method to load the model weights.

DonutSwin Model transformer with an image classification head on top (a linear layer on top of the final hidden state of the [CLS] token) e.g. for ImageNet.

> Note that it's possible to fine-tune DonutSwin on higher resolution images than the ones it has been trained on, by setting `interpolate_pos_encoding` to `True` in the forward of the model. This will interpolate the pre-trained position embeddings to the higher resolution.

This model inherits from PreTrainedModel. Check the superclass documentation for the generic methods the library implements for all its model (such as downloading or saving, resizing the input embeddings, pruning heads etc.)

This model is also a PyTorch torch.nn.Module subclass. Use it as a regular PyTorch Module and refer to the PyTorch documentation for all matter related to general usage and behavior.

forward <>

```
( pixel_values: torch.FloatTensor | None = None, labels: torch.LongTensor | None = None,
output_attentions: bool | None = None, output_hidden_states: bool | None = None,
interpolate_pos_encoding: bool = False, return_dict: bool | None = None, **kwargs ) →
DonutSwinImageClassifierOutput or tuple(torch.FloatTensor)
```

Expand 6 parameters

The DonutSwinForImageClassification forward method, overrides the `__call__` special method.

> Although the recipe for forward pass needs to be defined within this function, one should call the `Module` instance afterwards instead of this since the former takes care of running the pre and post processing steps while the latter silently ignores them.

- **loss** (`torch.FloatTensor` of shape `(1,)`, *optional*, returned when `labels` is provided) — Classification (or regression if config.num_labels==1) loss.
- **logits** (`torch.FloatTensor` of shape `(batch_size, config.num_labels)`) — Classification (or regression if config.num_labels==1) scores (before SoftMax).
- **hidden_states** (`tuple[torch.FloatTensor, ...]`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) — Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the model has an embedding layer, + one for the output of each layer) of shape `(batch_size, sequence_length, hidden_size)`.

  Hidden-states of the model at the output of each layer plus the optional initial embedding outputs.
- **attentions** (`tuple[torch.FloatTensor, ...]`, *optional*, returned when `output_attentions=True` is passed or when `config.output_attentions=True`) — Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size, num_heads, sequence_length, sequen`

Attentions weights after the attention softmax, used to compute the weighted average in the self-attention heads.

- **reshaped_hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) — Tuple of `torch.FloatTensor` (one for the output of the embeddings + one for the output of each stage) of shape `(batch_size, hidden_size, height, width)`.

  Hidden-states of the model at the output of each layer plus the initial embedding outputs reshaped to include the spatial dimensions.

Example:

```
>>> from transformers import AutoImageProcessor, DonutSwinForImageClassification
>>> import torch
>>> from datasets import load_dataset

>>> dataset = load_dataset("huggingface/cats-image")
>>> image = dataset["test"]["image"][0]

>>> image_processor = AutoImageProcessor.from_pretrained("naver-clova-ix/donut-bas
>>> model = DonutSwinForImageClassification.from_pretrained("naver-clova-ix/donut-

>>> inputs = image_processor(image, return_tensors="pt")

>>> with torch.no_grad():
...     logits = model(**inputs).logits

>>> # model predicts one of the 1000 ImageNet classes
>>> predicted_label = logits.argmax(-1).item()
>>> print(model.config.id2label[predicted_label])
...
```

Update on GitHub

← DePlot EdgeTAM →